

Final Mock Exam — Set B — Solutions

Lectures 1–12 (comprehensive)

Prof. Dr. Christoph Weisser

Quantitative Research Methods · HSBI

Summer Semester 2026

Overview of the final mock exam — Set B

Problem	Topic	Lecture slides	Points
P1	Splines (beyond linearity)	Chapter 7 — Moving Beyond Linearity (Lecture 9)	16
P2	Generalised additive models	Chapter 7 — Moving Beyond Linearity (Lecture 9)	8
P3	Trees by hand	Chapter 8 — Tree-Based Methods (Lecture 10)	20
P4	Ensembles (bagging/RF/boosting)	Chapter 8 — Tree-Based Methods (Lecture 10)	16
P5	Neural networks by hand	Chapter 10 — Deep Learning (Lecture 11)	20
P6	Multiple testing	Chapter 13 — Multiple Testing (Lecture 12)	20
P7	Cumulative essentials	Chapters 2–6 (Lectures 1–8)	20
Σ			120

Exam conditions

120 minutes for 120 points (≈ 1 point per minute). Allowed aids: non-programmable calculator and one handwritten A4 sheet (both sides). All required constants (e-powers and logs) are given in the problems. Justify every answer.

Problem 1 — task

Problem 1 (16 P) — Moving beyond linearity: splines

Lecture slides: Chapter 7 — Moving Beyond Linearity (Lecture 9).

Throughout: regression splines on a single predictor X with $K = 5$ interior knots.

- (a) [4 P] Degrees of freedom of a cubic spline with K interior knots — justify by counting parameters and constraints; evaluate for $K = 5$.
- (b) [4 P] Degrees of freedom of a *natural* cubic spline with $K = 5$ knots; which extra constraint acts where, and why is it useful?
- (c) [4 P] Smoothing spline $\sum_i (y_i - g(x_i))^2 + \lambda \int g''(t)^2 dt$: role of the two terms; behaviour of $\hat{g}$ as $\lambda \rightarrow 0$ and $\lambda \rightarrow \infty$.
- (d) [4 P] Truncated-power basis of a cubic spline with one knot ξ ; define $(x - \xi)_+^3$; why does adding it keep the fit smooth at ξ ?

Problem 1 — solution (a): df of a cubic spline

Solution P1(a) — counting parameters minus constraints

Step 1 — parameters. With K interior knots the range of X is cut into $K + 1$ sub-intervals; on each the fit is a *separate* cubic with 4 coefficients:

$$\underbrace{4(K + 1)}_{4 \text{ coeffs} \times (K+1) \text{ pieces}} = 4K + 4 \quad \text{free parameters (before constraints).}$$

Step 2 — smoothness constraints. At each of the K knots the two adjacent cubics must agree in *value, first and second derivative* (g, g', g'' continuous): 3 constraints per knot, $3K$ in total.

$$\text{df} = 4(K + 1) - 3K = 4K + 4 - 3K = \boxed{K + 4} \quad \xrightarrow{K=5} \quad \text{df} = 5 + 4 = 9.$$

Basis check. The truncated-power basis uses $1, x, x^2, x^3$ (four functions) *plus* one term $(x - \xi_k)_+^3$ per knot, giving $4 + K$ functions — the same count. *Read-off:* the df grow *linearly* in the knots (one extra df per additional knot).

Problem 1 — solution (b): natural cubic spline

Solution P1(b) — the boundary constraint

Extra constraint. A *natural* cubic spline is additionally required to be *linear beyond the two boundary knots*, i.e. on the outer regions $X < \xi_1$ and $X > \xi_K$.

Cost of the constraint. Linearity forces the quadratic *and* cubic terms to vanish in *each* outer region: that removes 2 parameters per side, 4 in total.

$$\text{df} = (K + 4) - 4 = \boxed{K} \quad \xrightarrow{K=5} \quad \text{df} = 5.$$

Why practitioners like it. Ordinary polynomials and unconstrained splines are most *erratic near the boundaries*, exactly where the data are sparse. Pinning the fit to a straight line there sharply cuts the *variance* of predictions at and beyond the outer knots, at the price of only a little extra bias — a favourable trade.

Problem 1 — solution (c): smoothing spline

Solution P1(c) — the two terms and the limits of λ

$$\hat{g} = \arg \min_g \underbrace{\sum_{i=1}^n (y_i - g(x_i))^2}_{(1) \text{ fit to data (RSS)}} + \lambda \underbrace{\int g''(t)^2 dt}_{(2) \text{ roughness / curvature}}$$

Meaning. Term (1) rewards closeness to the observations; g'' is the local *curvature*, so $\int g''^2$ is large for wiggly g — term (2) penalises wiggleness. $\lambda \geq 0$ tunes the trade-off.

- $\lambda \rightarrow 0$: penalty switched off $\Rightarrow \hat{g}$ *interpolates* every training point (training RSS = 0), maximally wiggly — overfitting; effective df $\rightarrow n$.
- $\lambda \rightarrow \infty$: any curvature is infinitely expensive $\Rightarrow g'' \equiv 0 \Rightarrow \hat{g}$ is the *least-squares straight line*; effective df $\rightarrow 2$.

In between, λ (chosen by LOOCV) moves the effective df smoothly from 2 up to n .

Problem 1 — solution (d): truncated-power basis

Solution P1(d) — one knot at ξ

Representation and basis function.

$$f(x) = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \beta_4 (x - \xi)_+^3, \quad (x - \xi)_+^3 = \begin{cases} (x - \xi)^3, & x > \xi \\ 0, & x \leq \xi. \end{cases}$$

Why smoothness survives. Differentiate the added term:

$$\frac{d}{dx}(x - \xi)_+^3 = 3(x - \xi)_+^2, \quad \frac{d^2}{dx^2}(x - \xi)_+^3 = 6(x - \xi)_+,$$

and all three of $(x - \xi)_+^3$, its first and its second derivative equal 0 at $x = \xi$. Hence adding $\beta_4(x - \xi)_+^3$ changes only the *third* derivative across the knot: the fit stays continuous with continuous g' and g'' — exactly the definition of a cubic spline.

Problem 2 — task

Problem 2 (8 P) — Generalised additive models

Lecture slides: Chapter 7 — Moving Beyond Linearity (Lecture 9).

- (a) [3 P] General form of a GAM for quantitative Y ; how does it generalise multiple linear regression?
- (b) [3 P] One interpretability advantage concerning the f_j ; the main structural limitation — and its repair.
- (c) [2 P] Backfitting in one or two sentences.

Problem 2 — solution (a): the GAM model form

Solution P2(a) — general form and how it generalises MLR

General form of a GAM for a quantitative response:

$$Y = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p) + \varepsilon = \beta_0 + \sum_{j=1}^p f_j(X_j) + \varepsilon.$$

How it generalises multiple linear regression. MLR is the *special case* in which every component is a straight line,

$$f_j(X_j) = \beta_j X_j \quad \implies \quad Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \varepsilon.$$

A GAM replaces each linear term by a flexible *smooth* function f_j (e.g. a natural or smoothing spline, or a local regression), so each predictor may act non-linearly — while the *additive* structure (the effects simply add up) is retained.

Problem 2 — solution (b): advantage, limitation, repair

Solution P2(b) — interpretability vs. interactions

Interpretability advantage. Because the effects are *added*, the contribution of each predictor is carried by its own function f_j . Each f_j can be *plotted and read off one panel at a time* with the other predictors held fixed — a non-linear effect is then as easy to inspect as a coefficient in a linear model (one curve per predictor).

Central limitation. Pure additivity rules out *interactions*: the effect of X_j is not allowed to depend on the value of another predictor X_k .

Repair. Add interaction components *by hand*, e.g. a product term $X_j \times X_k$, or a low-dimensional smooth surface $f_{jk}(X_j, X_k)$, as an extra additive block in the model.

Problem 2 — solution (c): the backfitting algorithm

Solution P2(c) — fitting one function at a time

Backfitting fits the many functions f_j by a simple *coordinate-wise* loop:

- 1 **Initialise** $\hat{\beta}_0 = \bar{y}$ and $\hat{f}_j \equiv 0$ for all j .
- 2 **Cycle** over $j = 1, \dots, p$: form the *partial residuals*

$$r_i = y_i - \hat{\beta}_0 - \sum_{k \neq j} \hat{f}_k(x_{ik}),$$

i.e. remove the currently-fitted effects of all *other* predictors, then re-estimate $\hat{f}_j$ by smoothing r_i against X_j (all other functions held fixed).

- 3 **Repeat** the cycles until the $\hat{f}_j$ no longer change appreciably (convergence).

Each update is thus just a one-dimensional smooth, which is what makes fitting a GAM tractable.

Problem 3 — task

Problem 3 (20 P) — Trees by hand

Lecture slides: Chapter 8 — Tree-Based Methods (Lecture 10).

Delivery vans x and parcels/day y (1,000s) for $n = 6$ depots; one split at s : $R_1 = \{x < s\}$ and $R_2 = \{x \geq s\}$ with region-mean predictions.

i	1	2	3	4	5	6
x_i	10	12	14	16	18	20
y_i	6	4	5	11	13	12

(a) **[8 P]** Evaluate the cutpoints $s = 15$ and $s = 17$ by total RSS; which is chosen? (b) **[2 P]** With the chosen split $s = 15$ (R_1 mean 5, R_2 mean 12): prediction for a new depot with $x = 13$. (c) **[6 P]** Node with 12 obs: 6 “Premium” / 3 “Standard” / 3 “Basic”: proportions; Gini; entropy ($\ln 0.5 = -0.6931$; $\ln 0.25 = -1.3863$); predicted class. (d) **[4 P]** Cost-complexity pruning: role of α in $\sum_m \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$.

Problem 3 — solution (a.1): cutpoint $s = 15$

Solution P3(a) part 1 — setup and cutpoint 15

What to minimise. For a single split at s , $RSS(s) = \sum_{m=1}^2 \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2$, each region predicting its own mean. (For reference the un-split root has $\bar{y} = 51/6 = 8.5$ and $RSS = 77.5$.)

Cutpoint $s = 15$. $R_1 = \{x < 15\} = \{\text{depots } 1, 2, 3\}$ with $y = \{6, 4, 5\}$; $R_2 = \{x \geq 15\} = \{\text{depots } 4, 5, 6\}$ with $y = \{11, 13, 12\}$:

$$\hat{y}_{R_1} = \frac{6+4+5}{3} = \frac{15}{3} = 5, \quad \hat{y}_{R_2} = \frac{11+13+12}{3} = \frac{36}{3} = 12.$$

$$RSS_{R_1} = (6 - 5)^2 + (4 - 5)^2 + (5 - 5)^2 = 1 + 1 + 0 = 2,$$

$$RSS_{R_2} = (11 - 12)^2 + (13 - 12)^2 + (12 - 12)^2 = 1 + 1 + 0 = 2 \Rightarrow RSS(15) = 2 + 2 = 4.$$

Problem 3 — solution (a.2): cutpoint $s = 17$ and the choice

Solution P3(a) part 2 — cutpoint 17 and the greedy decision

Cutpoint $s = 17$. $R_1 = \{x < 17\} = \{\text{depots } 1, 2, 3, 4\}$ with $y = \{6, 4, 5, 11\}$; $R_2 = \{x \geq 17\} = \{\text{depots } 5, 6\}$ with $y = \{13, 12\}$:

$$\hat{y}_{R_1} = \frac{6+4+5+11}{4} = \frac{26}{4} = 6.5, \quad \hat{y}_{R_2} = \frac{13+12}{2} = 12.5.$$

$$\text{RSS}_{R_1} = (6-6.5)^2 + (4-6.5)^2 + (5-6.5)^2 + (11-6.5)^2 = 0.25 + 6.25 + 2.25 + 20.25 = 29,$$

$$\text{RSS}_{R_2} = (13-12.5)^2 + (12-12.5)^2 = 0.25 + 0.25 = 0.5 \Rightarrow \text{RSS}(17) = 29.5.$$

Decision. Recursive binary splitting is *greedy* and picks the smaller RSS: $4 \ll 29.5 \Rightarrow \boxed{s = 15}$. It cleanly separates the three low depots (mean 5) from the three high ones (mean 12); $s = 17$ lumps the high depot $y = 11$ in with the lows, alone costing $(11 - 6.5)^2 = 20.25$.

Problem 3 — solution (b): prediction for a new depot

Solution P3(b) — routing a new depot down the tree

Prediction rule of a tree. Send the new observation down the tree by its split rules until it lands in a leaf; predict that leaf's region mean.

The chosen split is at $s = 15$ with $\hat{y}_{R_1} = 5$ ($x < 15$) and $\hat{y}_{R_2} = 12$ ($x \geq 15$). For the new depot with $x = 13$:

$$x = 13 < 15 \Rightarrow \text{region } R_1 \Rightarrow \hat{y} = \hat{y}_{R_1} = 5 \quad (\text{i.e. 5,000 parcels per day}).$$

Note. The tree is *piecewise constant*: every depot with $x < 15$ gets the same prediction 5, regardless of the exact number of vans — $x = 13$ is not interpolated between neighbouring points.

Problem 3 — solution (c): node impurity (Gini and entropy)

Solution P3(c) — three classes in one node

Class proportions (12 obs: 6 Premium, 3 Standard, 3 Basic):

$$\hat{p}_{\text{Prem}} = \frac{6}{12} = 0.5, \quad \hat{p}_{\text{Std}} = \frac{3}{12} = 0.25, \quad \hat{p}_{\text{Basic}} = \frac{3}{12} = 0.25.$$

Gini index $G = \sum_k \hat{p}_k(1 - \hat{p}_k)$:

$$G = 0.5(1 - 0.5) + 0.25(1 - 0.25) + 0.25(1 - 0.25) = 0.25 + 0.1875 + 0.1875 = 0.625.$$

Entropy $D = -\sum_k \hat{p}_k \ln \hat{p}_k$ (with $\ln 0.5 = -0.6931$, $\ln 0.25 = -1.3863$):

$$D = -(0.5(-0.6931) + 0.25(-1.3863) + 0.25(-1.3863)) = 0.3466 + 0.6931 = 1.040.$$

Predicted class = the majority, **Premium** (estimated probability 0.5). Both G and D vanish only for a *pure* node; the sizeable values here reflect a genuinely impure node with three classes present.

Problem 3 — solution (d): cost-complexity pruning

Solution P3(d) — the role of α

$$\sum_{m=1}^{|T|} \sum_{i: x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T| \quad (\alpha \geq 0 : \text{price per terminal node}).$$

α trades the subtree's *fit* (first term) against its *size* $|T|$:

- $\alpha = 0$: the criterion is just the training RSS $\Rightarrow$ the full unpruned tree T_0 is optimal (size is free).
- $\alpha \uparrow$: every extra leaf must “earn its keep”; a branch whose RSS reduction is smaller than α is pruned away, giving a *nested sequence* of ever-smaller subtrees (weakest-link pruning).

Choosing α in practice. Use *cross-validation*: compute CV error along the nested sequence, pick the α with the lowest CV error, then refit that subtree on the full data — balancing bias against variance rather than merely minimising training error.

Problem 4 — task

Problem 4 (16 P) — Ensembles

Lecture slides: Chapter 8 — Tree-Based Methods (Lecture 10).

- (a) **[6 P]** B identically distributed trees with variance σ^2 and pairwise correlation ρ :
 $\text{Var}(\frac{1}{B} \sum_b \hat{f}_b(x)) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. (i) Why does this show variance reduction and what limits it? (ii) Evaluate for $B = 500$ and $\rho = 0.7$ and $\sigma^2 = 4$; compare with one tree. (iii) Limit as $B \rightarrow \infty$?
- (b) **[4 P]** Random forests: modification at each split; recommended m for classification with $p = 36$; why does it improve on bagging (use the variance formula $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ from (a))?
- (c) **[4 P]** A boosting model in `scikit-learn`:
`GradientBoostingRegressor(n_estimators=500, learning_rate=0.01, max_depth=1)`
state what each of the three arguments controls and the slow-learning idea.
- (d) **[2 P]** What does the OOB error estimate and why is it free?

Problem 4 — solution (a.1): why bagging reduces variance

Solution P4(a)(i) — reading the variance formula

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)\right) = \underbrace{\rho \sigma^2}_{\text{floor, no } B} + \underbrace{\frac{1-\rho}{B} \sigma^2}_{\rightarrow 0 \text{ as } B \uparrow}.$$

Variance reduction. The second term $\frac{1-\rho}{B} \sigma^2$ shrinks to 0 as B grows: averaging over many trees removes the *independent part* of their variability. This is precisely the variance reduction that bagging delivers.

What limits it. The first term $\rho \sigma^2$ does *not* depend on B . Bootstrap trees are grown on overlapping resamples of the same data and share the same strong predictors, so they are *positively correlated* ($\rho > 0$); that common variance cannot be averaged away. It is the *floor* that caps how far bagging can reduce variance.

Problem 4 — solution (a.2): numerical evaluation and limit

Solution P4(a)(ii)–(iii) — numerical value and the limit

(ii) **Variance of the bagged average.**

$$\text{Var} = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 = 0.7 \cdot 4 + \frac{1-0.7}{500} \cdot 4 = 2.8 + \frac{0.3 \cdot 4}{500} = 2.8 + 0.0024 = 2.802.$$

Compared with a single tree ($\sigma^2 = 4$), the bagged average has variance 2.802 — a reduction of about 30%.

(iii) **Limit** $B \rightarrow \infty$. The vanishing term disappears and the variance approaches the floor

$$\text{Var} \longrightarrow \rho\sigma^2 = 0.7 \cdot 4 = 2.8.$$

Adding still more trees never overfits; it only *stabilises* the average and cannot push the variance below the floor $\rho\sigma^2$.

Problem 4 — solution (b): random forests

Solution P4(b) — decorrelating the trees

Modification at each split. A random forest may consider only a *fresh random subset of m out of the p predictors* as candidates for *each* split (a new draw at every node).

Recommended m (classification). $m \approx \sqrt{p}$, so for $p = 36$:

$$m = \sqrt{36} = 6.$$

Why it beats bagging (via the formula). In plain bagging a dominant predictor is chosen at the top split of nearly every tree, making the trees look alike and pushing the correlation ρ *up*. Withholding that predictor from about $(p - m)/p$ of the split searches *decorrelates* the trees, i.e. lowers ρ . In $\text{Var} = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$ a smaller ρ lowers the irreducible floor $\rho\sigma^2$ — the part of the variance that plain averaging can never remove.

Problem 4 — solution (c): boosting tuning parameters

Solution P4(c) — reading the GradientBoostingRegressor call

- `n_estimators`= $B = 500$: the number of small trees added *in sequence*. Unlike bagging, boosting *can* overfit if B is too large, so B is tuned by cross-validation.
- `learning_rate`= $\lambda = 0.01$ (shrinkage): scales the contribution of each new tree, i.e. the speed of learning; a smaller λ usually needs a larger B .
- `max_depth`= $d = 1$ (interaction depth): the number of splits per tree, hence the interaction order captured; $d = 1$ means *stumps* — an additive model.

The “slow learning” idea. Each new tree is fit to the *current residuals*, and only the small fraction λ of it is added to the model. The fit therefore improves *gradually and precisely where it is still weak*, rather than fitting the data hard in a single step — which typically generalises better.

Problem 4 — solution (d): out-of-bag error

Solution P4(d) — a free test-error estimate

The out-of-bag idea. Each bootstrap sample omits, on average, about a third of the observations:

$$\Pr(\text{obs left out of a given bag}) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n \rightarrow \infty} e^{-1} \approx 0.368.$$

What OOB estimates. Predict each observation using *only* the trees that did **not** train on it, then aggregate; the resulting error is the **OOB error**, a valid estimate of the bagged model's *test* error.

Why it is essentially free. It reuses a by-product of the bootstrap already computed during training — no separate validation set and no extra cross-validation loop are needed.

Problem 5 — task

Problem 5 (20 P) — Neural networks by hand

Lecture slides: Chapter 10 — Deep Learning (Lecture 11).

Input $x = (2,1)$; hidden ReLU units $g(z) = \max\{0,z\}$; linear output:

$$A_1 = g(-2 + 3x_1 - 1x_2) \quad A_2 = g(1 - 2x_1 + 1x_2) \quad f(x) = -1 + 2A_1 + 3A_2$$

- (a) **[8 P]** Forward pass step by step: pre-activations; A_1 and A_2 ; output $f(x)$. What did the ReLU do to unit 2 and why are non-linear activations essential?
- (b) **[6 P]** Logits $z = (2.0, 1.0, -1.0)$; true class 1: all three softmax probabilities and the cross-entropy loss ($e^2 = 7.389$; $e^1 = 2.718$; $e^{-1} = 0.368$; $\ln 10.475 = 2.349$).
- (c) **[4 P]** Parameter count of a $p = 6 \rightarrow k = 4 \rightarrow K = 3$ network with biases (show both layers).
- (d) **[2 P]** Conv layer: 32×32 input; $F = 5$; $P = 0$; $S = 1$: output size via $(W - F + 2P)/S + 1$.

Problem 5 — solution (a): the forward pass

Solution P5(a) — pre-activations then ReLU then output

Step 1 — pre-activations at $x = (x_1, x_2) = (2, 1)$:

$$z_1 = -2 + 3x_1 - 1x_2 = -2 + 3 \cdot 2 - 1 \cdot 1 = -2 + 6 - 1 = 3,$$

$$z_2 = 1 - 2x_1 + 1x_2 = 1 - 2 \cdot 2 + 1 \cdot 1 = 1 - 4 + 1 = -2.$$

Step 2 — ReLU activations $g(z) = \max\{0, z\}$:

$$A_1 = \max\{0, 3\} = 3, \quad A_2 = \max\{0, -2\} = 0.$$

Step 3 — linear output:

$$f(x) = -1 + 2A_1 + 3A_2 = -1 + 2 \cdot 3 + 3 \cdot 0 = -1 + 6 + 0 = 5.$$

Comment. The ReLU *switched unit 2 off* (negative z_2 clipped to 0), so it contributes nothing for this input — though it could for others. This thresholding is the source of non-linearity: with a purely linear g a network of *any* depth would collapse into a single linear function of x .

Problem 5 — solution (b): softmax and cross-entropy

Solution P5(b) — softmax then cross-entropy for true class 1

Exponentiate the logits and sum (softmax $\hat{p}_k = e^{z_k} / \sum_j e^{z_j}$):

$$e^{2.0} = 7.389, \quad e^{1.0} = 2.718, \quad e^{-1.0} = 0.368 \quad \Rightarrow \quad \sum = 7.389 + 2.718 + 0.368 = 10.475.$$

Softmax probabilities:

$$\hat{p}_1 = \frac{7.389}{10.475} = 0.705, \quad \hat{p}_2 = \frac{2.718}{10.475} = 0.259, \quad \hat{p}_3 = \frac{0.368}{10.475} = 0.035$$

(sum = 0.999 $\approx$ 1 $\checkmark$). **Cross-entropy loss** for true class 1:

$$-\ln \hat{p}_1 = -\ln \frac{e^2}{10.475} = -(2 - \ln 10.475) = \ln 10.475 - 2 = 2.349 - 2 = 0.349.$$

Sanity checks. A perfect prediction $\hat{p}_1 = 1$ gives loss 0; the uniform guess $\frac{1}{3}$ would give $\ln 3 \approx 1.099$. Here the network is right and fairly confident, so the loss (0.349) sits well below 1.099.

Problem 5 — solution (c): parameter count

Solution P5(c) — fully connected $6 \rightarrow 4 \rightarrow 3$ with biases

Rule. A dense layer from d_{in} to d_{out} units has $d_{\text{in}} \cdot d_{\text{out}}$ weights and d_{out} biases.

Layer	Weights	Biases	Parameters
input $\rightarrow$ hidden ($6 \rightarrow 4$)	$6 \cdot 4 = 24$	4	28
hidden $\rightarrow$ output ($4 \rightarrow 3$)	$4 \cdot 3 = 12$	3	15
total			43

$$(6 \cdot 4 + 4) + (4 \cdot 3 + 3) = 28 + 15 = 43 \text{ parameters.}$$

Biases matter: dropping them would leave only $24 + 12 = 36$; each unit's bias adds one more free parameter.

Problem 5 — solution (d): convolution output size

Solution P5(d) — convolution output size

$$\text{output width} = \frac{W - F + 2P}{S} + 1 = \frac{32 - 5 + 2 \cdot 0}{1} + 1 = \frac{27}{1} + 1 = 28.$$

By symmetry the height is also 28, so the output feature map is 28×28 .

Intuition. With no padding ($P = 0$) a 5×5 filter cannot be centred on the outer ring of pixels, so each spatial dimension shrinks by $F - 1 = 4$ (from 32 to 28). Padding ($P > 0$) would add a border to preserve size; a larger stride S would shrink it further.

Problem 6 — task

Problem 6 (20 P) — Multiple testing

Lecture slides: Chapter 13 — Multiple Testing (Lecture 12).

- (a) **[4 P]** $m = 10$ independent tests at $\alpha = 0.05$; all nulls true: define and compute the FWER $((0.95)^{10} = 0.5987)$; interpret.
- (b) **[7 P]** Six ordered p -values 0.002; 0.009; 0.012; 0.015; 0.040; 0.300 — FWER control at $\alpha = 0.05$ with (i) Bonferroni and (ii) Holm; thresholds; rejected hypotheses; why is Holm never weaker than Bonferroni?
- (c) **[6 P]** Benjamini–Hochberg at $q = 0.05$ on the same six p -values (0.002; 0.009; 0.012; 0.015; 0.040; 0.300; $m = 6$): thresholds jq/m ; rejections; what exactly does FDR control guarantee?
- (d) **[3 P]** Confirmatory Phase-III trial with $m = 4$ pre-specified endpoints (one hit could support approval): FWER or FDR? Justify.

Problem 6 — solution (a): FWER of 10 independent tests

Solution P6(a) — definition and computation

Definition. The family-wise error rate is the probability of *at least one* false rejection,

$$\text{FWER} = \Pr(V \geq 1), \quad V = \#\{\text{true nulls that are falsely rejected}\}.$$

Computation. All 10 nulls true, tests independent, each at level 0.05:

$$\text{FWER} = 1 - \Pr(\text{no false rejection}) = 1 - (1 - 0.05)^{10} = 1 - (0.95)^{10} = 1 - 0.5987 = 0.4013.$$

Interpretation. Although each single test errs with probability only 5%, the chance of *at least one* spurious “discovery” across ten unadjusted tests is about 40% — testing many hypotheses without adjustment makes false findings very likely.

Problem 6 — solution (b.1): Bonferroni

Solution P6(b)(i) — Bonferroni at the 0.05 level

Rule. Reject $H_{(j)}$ iff $p_{(j)} \leq \frac{\alpha}{m} = \frac{0.05}{6} = 0.00833$ — the *same* flat threshold for every test.

Apply to the ordered p -values:

$$p_{(1)} = 0.002 \leq 0.00833 \checkmark, \quad p_{(2)} = 0.009 > 0.00833 \times, \dots$$

Only the smallest passes, so Bonferroni rejects $H_{(1)}$ alone: **1 rejection**.

Remark. $p_{(2)} = 0.009$ is well below the naive single-test cutoff 0.05, yet Bonferroni does *not* reject it — the price of controlling the FWER with one strict flat threshold is low power.

Problem 6 — solution (b.2): Holm step-down

Solution P6(b)(ii) — Holm and why it dominates Bonferroni

Rule. In order, compare $p_{(j)}$ with $\frac{\alpha}{m-j+1}$ and *stop at the first failure*; reject everything before the stop.

j	1	2	3	4	5	6
$p_{(j)}$	0.002	0.009	0.012	0.015	0.040	0.300
$\alpha/(m-j+1)$	0.00833	0.0100	0.0125	0.0167	0.0250	0.0500
decision	✓	✓	✓	✓	stop	—

$0.015 \leq 0.0167$ passes, but $0.040 > 0.0250$ fails $\Rightarrow$ reject $H_{(1)}, \dots, H_{(4)}$: **4 rejections**.

Why Holm never rejects fewer than Bonferroni. Holm's thresholds satisfy $\frac{\alpha}{m-j+1} \geq \frac{\alpha}{m}$ for all j (equality only at $j = 1$), so any p -value that clears the strict Bonferroni bar also clears Holm's more lenient later hurdles. Holm is therefore *uniformly at least as powerful*, while still controlling the FWER with no independence assumption.

Problem 6 — solution (c): Benjamini–Hochberg

Solution P6(c) — Benjamini–Hochberg at FDR 0.05

Thresholds $jq/m = j \cdot 0.05/6$; then find the *largest* j with $p_{(j)} \leq jq/m$:

j	1	2	3	4	5	6
$p_{(j)}$	0.002	0.009	0.012	0.015	0.040	0.300
jq/m	0.00833	0.0167	0.0250	0.0333	0.0417	0.0500
$p_{(j)} \leq jq/m?$	yes	yes	yes	yes	yes	no

Largest crossing index $L = 5$ ($0.040 \leq 0.0417$, while $0.300 > 0.0500$). BH rejects *all* $H_{(j)}$ with $j \leq L$, i.e. $H_{(1)}, \dots, H_{(5)}$: **5 rejections**.

What FDR control guarantees. $\text{FDR} = E\left[\frac{V}{\max(R, 1)}\right] \leq q = 0.05$ — *on average* at most 5% of the rejected hypotheses are false discoveries. It does *not* promise that at most 5% of these particular 5 are false, nor does it bound $\Pr(V \geq 1)$ (that would be FWER control).

Problem 6 — solution (d): FWER or FDR for a Phase-III trial?

Solution P6(d) — confirmatory trial recommendation

Recommendation: control the FWER.

Why. In a confirmatory trial a *single* false positive is very costly — it could support approving an *ineffective* drug — so the study must guard against *any* Type I error, not merely keep their average proportion small (which is all FDR does).

Cost is low here. With only $m = 4$ pre-specified endpoints an FWER procedure (e.g. Holm, or Bonferroni at $0.05/4 = 0.0125$) sacrifices little power. FDR control is designed for large *exploratory* screens where a few false leads are tolerable; it is too permissive for a confirmatory, decision-driving trial.

Problem 7 — task

Problem 7 (20 P) — Cumulative essentials

Lecture slides: Chapters 2–6 (Lectures 1–8).

- (a) **[4 P]** `scikit-learn`'s `confusion_matrix` (labels $[0, 1]$; rows true, cols predicted) returns $\begin{bmatrix} 120 & 30 \\ 10 & 40 \end{bmatrix}$: compute error rate, sensitivity, specificity, precision (positive class 1); which number traces the ROC curve and how do sensitivity/specificity move?
- (b) **[4 P]** KNN: (i) predict at $x_0 = 5$ with $k = 3$ from $(1,3), (4,6), (6,8), (7,7), (10,12)$; (ii) how k drives the bias–variance trade-off; most flexible extreme; why is 1-NN training error 0?
- (c) **[4 P]** k -fold CV: what it does; one reason over LOOCV; one over a single validation split.
- (d) **[2 P]** $p = 2,000$ predictors, only a handful matter: ridge or lasso? Where does PCR sit?
- (e) **[6 P]** Match {linear regression; lasso; logistic regression; random forest} — each once: (i) \$/schooling-year with CI; (ii) BP from 8,000 genes ($n=150$) with a short gene list; (iii) spam yes/no with odds for auditors; (iv) machine failure from interacting sensors, accuracy only.

Problem 7 — solution (a): reading the confusion matrix

Solution P7(a) — reading the confusion matrix

Read the array (rows = true, cols = predicted, order [0, 1]; diagonal = correct):

$$TN = 120, \quad FP = 30, \quad FN = 10, \quad TP = 40 \quad (n = 200 : 150 \text{ healthy, } 50 \text{ diseased}).$$

$$\text{error} = \frac{FP + FN}{200} = \frac{30 + 10}{200} = 0.200, \quad \text{sensitivity} = \frac{TP}{TP + FN} = \frac{40}{50} = 0.800,$$

$$\text{specificity} = \frac{TN}{TN + FP} = \frac{120}{150} = 0.800, \quad \text{precision} = \frac{TP}{TP + FP} = \frac{40}{70} = 0.571.$$

ROC curve. Varying the *classification threshold* on the predicted probability traces the ROC curve: *lowering* it labels more patients “diseased”, raising sensitivity (fewer misses) but lowering specificity (more false alarms); raising it does the reverse.

Problem 7 — solution (b): K -nearest-neighbours

Solution P7(b) — prediction and the bias–variance trade-off

(i) **Predict at $x_0 = 5$ with $k = 3$.** Distances $|x_i - 5|$:

$$|1 - 5| = 4, |4 - 5| = 1, |6 - 5| = 1, |7 - 5| = 2, |10 - 5| = 5.$$

The three nearest are $x = 4, 6, 7$ with responses $y = 6, 8, 7$, so

$$\hat{y}(5) = \frac{6 + 8 + 7}{3} = \frac{21}{3} = 7.$$

(ii) **Role of k .** Small $k \Rightarrow$ very flexible fit: *low bias, high variance* (the prediction hangs on a few nearby points). Large $k \Rightarrow$ smooth, stable fit: *high bias, low variance*. Flexibility $\propto 1/k$, so the most flexible extreme is $k = 1$. The *training* error of 1-NN is always 0 because each point's nearest neighbour is *itself* — it predicts its own response exactly, so training error is no guide to test performance.

Problem 7 — solution (c): k -fold cross-validation

Solution P7(c) — k -fold vs. LOOCV vs. a single split

What it does. Split the data into k roughly equal folds; each fold serves *once* as the validation set while the model trains on the other $k - 1$ folds; average the k resulting errors.

Prefer $k = 5$ or 10 over LOOCV. The n leave-one-out fits use almost identical training sets, so their errors are highly *correlated* and their average has *higher variance* than 5/10-fold CV (and LOOCV costs n fits, absent a shortcut formula).

Prefer it over a single validation split. One split is highly variable (depends on the particular partition) and biased *upward*, since only part of the data trains the model; k -fold trains on $(k - 1)/k$ of the data and averages k estimates $\Rightarrow$ more stable and less biased.

Problem 7 — solution (d): lasso vs. ridge and PCR

Solution P7(d) — lasso for a sparse high-dimensional problem

Choose the lasso. The ℓ_1 penalty drives many coefficients *exactly to zero* and thus performs *variable selection* — ideal when the truth is *sparse* (a few real signals among 2,000 predictors) and an interpretable small model is wanted.

Why not ridge. The ℓ_2 penalty only *shrinks* coefficients; it keeps all 2,000 predictors and never sets any exactly to zero.

Where PCR sits. PCR is likewise *not* sparse: it regresses on a few principal components, and *all* original variables load on those components, so no variable is truly dropped. Like ridge, PCR performs no selection — neither delivers the short interpretable list that the lasso does.

Problem 7 — solution (e): matching methods to tasks

Solution P7(e) — each method used exactly once

- (i) **Linear regression** — quantitative response with $n \gg p$ and the goal of *inference*: an effect size (£ per schooling-year) *with a confidence interval*, exactly what OLS standard errors provide.
- (ii) **Lasso** — $p = 8,000 \gg n = 150$ makes OLS infeasible; the ℓ_1 penalty regularises the fit *and* returns a short list of relevant markers.
- (iii) **Logistic regression** — binary response with interpretable coefficients: $e^{\hat{\beta}_j}$ are *odds ratios*, ideal for explaining odds to auditors.
- (iv) **Random forest** — automatically captures non-linearities and interactions and is accurate with many mixed predictors; interpretability is not required.

One-line rule of thumb

Inference on few predictors $\rightarrow$ linear/logistic regression; sparse high-dimensional selection $\rightarrow$ lasso; pure predictive accuracy with complex structure $\rightarrow$ random forest.